# Assignment 2

Andres Aranguren (2839590)

November 2024

## 1 Answers

**question 1** Let f(X, Y) = X / Y for two matrices X and Y (where the division is element-wise). Derive the backward for X and for Y . Show the derivation.

We want to obtain $\frac{\partial l}{\partial x}$ and $\frac{\partial l}{\partial y}$.

First let us define:

$$Z_{ij} = \frac{X_{ij}}{Y_{ij}} \tag{1}$$

We know that the backward operation computes the graident of the loss with respect to inputs given the gradient of the loss with respect to output.

$$\frac{\partial l}{\partial X} = \frac{\partial l}{\partial Z} \cdot \frac{\partial Z}{\partial X} \tag{2}$$

$$\frac{\partial Z}{\partial X} = \frac{1}{Y_{ij}} \tag{3}$$

$$\frac{\partial l}{\partial Z} = L\,' \tag{4}$$

Derivative of loss function l with respect to the output Z.
Replacing Eq.(4) in Eq.(8)

$$\boxed{\frac{\partial l}{\partial X} = L\,' \cdot \frac{1}{Y_{ij}}} \tag{5}$$

$$\frac{\partial l}{\partial Y} = \frac{\partial l}{\partial Z} \cdot \frac{\partial Z}{\partial Y} \tag{6}$$

$$\frac{\partial Z}{\partial Y} = -\frac{X}{Y^2} \tag{7}$$

$$\boxed{\frac{\partial l}{\partial Y} = L\,' \cdot -\frac{X}{Y^2}} \tag{8}$$

**question 2** Show that whatever f is, the backward of F is the element-wise application of f prime applied to the elements of X, multiplied (element-wise) by the gradient of the loss with respect to the outputs. (For a concrete example thing of the sigmoid function from the lectures).

$$F'(x) = f'(x) \cdot L'(x) \tag{9}$$

$$F(X_{ij}) = f(X_{ij}) \tag{10}$$

$$y_{ij} = F(X_{ij}) \tag{11}$$

$$y_{ij} = f(X_{ij}) \tag{12}$$

Gradient of loss with respect to output

$$\frac{\partial l}{\partial y} = L' \tag{13}$$

Back-pass for one element $y_{ij} = f(x_{ij})$ with respect to $X_{ij}$ is:

$$\frac{\partial y_{ij}}{\partial X_{ij}} = f'(X_{ij}) \tag{14}$$

Based on chain-rule we have:

$$\boxed{\frac{\partial L}{\partial X_{ij}} = \frac{\partial L}{\partial y_{ij}} \cdot \frac{\partial y_{ij}}{\partial X_{ij}} = L' \cdot f'(X)} \tag{15}$$

Example with sigmoid function

1. Sigmoid function

$$f(x) = \frac{1}{1 + e^{-x}} \tag{16}$$

$$f'(x) = f(x) \cdot (1 - f(x)) \tag{17}$$

2. Compute tensor backward pass
   Based on Eq.(15)

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial F} \cdot \frac{\partial y_{ij}}{\partial X_{ij}} = L' \cdot F(X) \cdot (1 - F(X)) \tag{18}$$

**question 3** . Which matrix operation computes the layer outputs? Work out the backward for this operation, providing gradients for both W and X.

$$y = X \cdot W \tag{19}$$

Where X and W have dimensions $(n \times f)$, $(f \times m)$ respectively, resulting in $y$ with dimension $(n \times m)$

$$\frac{\partial L}{\partial y} = L\,'(X) \tag{20}$$

As described in Eq.(15)

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial W} \tag{21}$$

$$\frac{\partial y}{\partial W} = X \tag{22}$$

$$\boxed{\frac{\partial L}{\partial W} = X^T \cdot L\,'} \tag{23}$$

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial X} \tag{24}$$

$$\frac{\partial y}{\partial X} = W \tag{25}$$

$$\boxed{\frac{\partial L}{\partial X} = L\,' \cdot W^T} \tag{26}$$

**question 4** . Let f(x) = Y be a function that takes a vector x, and returns the matrix Y consisting of 16 columns that are all equal to x. Work out the backward of f.

$$x(d \times 1) \tag{27}$$

$$y = (d \times 16) \tag{28}$$

$$y = \begin{bmatrix} x^{(1)} & x^{(2)} & \cdots & x^{(16)} \end{bmatrix}$$

$$\begin{bmatrix} x_{1,1} & \cdots & x_{1,16} \\ \vdots & \ddots & \vdots \\ x_{d,1} & \cdots & x_{d,16} \end{bmatrix}$$

To compute the backward of f, $\frac{\partial L}{\partial y}$ we need to calculate $\frac{\partial L}{\partial x}$ given the y is a function of vector x

We know that $y_{ij} = x_i$ where $i = 1, 2, 3...d$ and $j = 1, 2, 3..16$ Thus to calculate

$\frac{\partial L}{\partial x_i}$, we calculate the contribution of each $x$ in row $i$ for all columns $j$ in the matrix $y$.

$$\frac{\partial L}{\partial x_i} = \sum_{j=1}^{16} \frac{\partial L}{\partial y_{ij}} \tag{29}$$

Since we apply the operation for the $i_{th}$ row we can rewrite as

$$\frac{\partial L}{\partial x_i} = \sum_{j=1}^{16} L\ '[:, j] \tag{30}$$

$$\frac{\partial L}{\partial x_i} = [1, 1, 1, 1, 1, 1...] \tag{31}$$

we obtain a vector of $(1 \times 16$ which sums for each contribution of the 16 columns of y.

**question 5** .

1. What does the c.value contain?
   c.value is a numpy array that contains the element-wise sum between the values of node a and b.

2. What does c.source refer to?
   c.source refers to the opNode that was used to create node c. In this case c was created using the add operation $+$.

3. What does c.source.inputs[0].value refer to? c.source.inputs[0] refers to the first input that was used to create c, since c = a+b then input[0] is tensor node a

4. What does a.grad refer to? What is its current value?
   a.grad refers to the initial matrix created to store the values of the gradient while computing the loss, its initialized as a zeros matrix with same dimension as the node in this case 3 by 3.

$$a.grad = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

**question 6** .

1. An OpNode is defined by its inputs, its outputs and the specific operation it represents (i.e.summation, multiplication). What kind of object defines this operation?
   The operation is defined by an abstract class, specifically Op. This type of class allows to create a set of methods that must be created within any child classes built form the abstract Op class. In this type of class methods are declared but not implemented.

```
    def __init__(self, op, context, inputs):
    super().__init__()
    #op is the operation class received as parameter
    self.op = op

class Op:

#Abstract class to represent an operation.

#Note that this is a class with only static methods. We never instantiate #it, we j
#the forward and backward methods in a way that we can refer to the op as #a whole.
#it has the following not implemented methods
    @staticmethod
    def forward(context : dict, *inputs, **kawrgs):

        pass

    @staticmethod
    def backward(context : dict, *goutput):

        pass
```

2. In which line of code is the actual addition performed?
   The addition of the two numpy arrays is performed in the following line.

```
    class Add(Op):
"""
Op for element-wise matrix addition.
"""
@staticmethod
def forward(context, a, b):
    assert a.shape == b.shape, f'Arrays not the same sizes ({a.shape} {b.shape}).'
    return a + b
```

   In section.A**Appendix** we present the sequence of calls in the code that lead to the given operation.

3. When an OpNode is created, its inputs are immediately set, together with a reference to the op that is being computed. The pointer to the output node(s) is left None at first. Why is this? In which line is the OpNode connected to the output nodes?
   The pointer to the output is left None at first since the value of the output depends on the forward result for a given operation for which we need the input values and a given operation.
   The OpNode is connected to the output nodes in the Op abstract class. More specifically in the doforward function, which manages the graph construction and refers to the forward method of the abstract class for each operation, computing the output tensor given a certain input.

Specific line

```
        # Connecting opnode to output
        opnode.outputs = outputs

    class Op:
    """
    Abstract class to represent an operation.

    Note that this is a class with only static methods. We never instantiate it, we jus
    the forward and backward methods in a way that we can refer to the op as a whole.

    """
    # extract the raw input values
        inputs_raw = [input.value for input in inputs]

        # compute the raw output values
        outputs_raw = cls.forward(context, *inputs_raw, **kwargs)

        if not type(outputs_raw) == tuple:
            outputs_raw = (outputs_raw, )

        opnode = OpNode(cls, context, inputs)

        outputs = [TensorNode(value=output, source=opnode) for output in outputs_raw]
        # Connecting opnode to output
        opnode.outputs = outputs
```

**question 7** The backward function of the relevant Ops is applied in the OpNode class, in the backward method, which calls the backward method of the Op class for a given operation.

```
class OpNode:
    def backward(self):
        ginputs_raw = self.op.backward(self.context, *goutputs_raw)
```

The class OpNode thus receives Op abstract class as a parameter and from it calls the specific backward for the operation. self.op.backward refers to:

```
class Op:
    @staticmethod
def backward(context : dict, *goutput):

    pass
```

The abstract class then refers to the implemented method of the backward function for a given operation. For example addition.

```python
class Add(Op):
"""
Op for element-wise matrix addition.
"""
@staticmethod
def forward(context, a, b):
    assert a.shape == b.shape, f'Arrays not the same sizes ({a.shape} {b.shape}).'
    return a + b
```

**question 8** Choose one of the ops Normalize, Expand, Select, Squeeze or Unsqueeze, and show that the implementation is correct. That is, for the given forward, derive the backward by hand (mathematically in LaTeX), and show that it matches what is implemented.
For this, we chose the ops Normalize which is defined by:

$$y_{ij} = \frac{x_{ij}}{\sum_k x_{ik}} \tag{32}$$

$k$ being the index to denote the sum that all includes all $k$ row of elements of row $i$.
For notation purpose let us refer to the sum as:

$$\sum_k x_{ik} = S_i \tag{33}$$

$$y_{ij} = \frac{x_{ij}}{S_i} \tag{34}$$

When computing $\frac{\partial y_{ij}}{\partial x_{ij}}$ we want to calculate the contribution of $x_{ij}$ in the output $y_{ij}$. Thus, when calculating:

$$\frac{\partial y_{ij}}{\partial x_{ik}} \tag{35}$$

We consider two cases

- When $j = k$, for example the contribution of $x_{11}$ in the output $y_{11}$
  We can apply quotient derivation rule to Eq.(38)

$$\frac{\partial y_{ij}}{\partial x_{ij}} = \frac{1 \cdot S_j - x_{ij} \cdot 1}{S_i^2} \tag{36}$$

$$\frac{\partial y_{ij}}{\partial x_{ij}} = \frac{S_j - x_{ij}}{S_i^2} \tag{37}$$

$$\frac{\partial y_{ij}}{\partial x_{ij}} = \frac{1}{S_i} - \frac{x_{ij}}{S_i^2} \tag{38}$$

- When $j \neq k$, for example, the contribution of $x_{12}$ in the output $y_{11}$.

$$\frac{\partial y_{ij}}{\partial x_{ij}} = \frac{0 \cdot S_j - x_{ij} \cdot 1}{S_i^2} \tag{39}$$

$$\frac{\partial y_{ij}}{\partial x_{ij}} = -\frac{x_{ij}}{S_i^2} \tag{40}$$

$$\frac{\partial l}{\partial x_{ij}} = \sum_k \frac{\partial l}{\partial y_{ik}} \cdot \frac{y_{ik}}{\partial x_{ij}} \tag{41}$$

As described in vugrad code we will treat the $\frac{\partial l}{\partial y_{ik}}$ as go

$$\frac{\partial l}{\partial x_{ij}} = \sum_k go \cdot \frac{y_{ik}}{\partial x_{ij}} \tag{42}$$

Hence for both cases we obtain:

- When $j = k$

$$\frac{\partial l}{\partial x_{ij}} = go_{ij} \cdot \left(\frac{1}{S_i} - \frac{x_{ij}}{S_i^2}\right) \tag{43}$$

- When $j \neq k$

$$\frac{\partial l}{\partial x_{ij}} = \sum_{k \neq j} go_{ik} \cdot \left(-\frac{x_{ij}}{S_i^2}\right) \tag{44}$$

Putting together both cases we obtain:

$$\frac{\partial l}{\partial x_{ij}} = \frac{go_{ij}}{S_i} - \frac{go_{ij} \cdot x_{ij}}{S_i^2} \tag{45}$$

This result matches the implemented derivation of the backward in the ops.py file for the normalization method

```
class Normalize(Op):
    @staticmethod
    def backward(context, go):

        x, sumd = context['x'], context['sumd']

        return (go / sumd) - ((go * x)/(sumd * sumd)).sum(axis=1, keepdims=True)
```

**question 9** The current network uses a Sigmoid nonlinearity on the hidden layer. Create an Op for a ReLU nonlinearity. Retrain the network. Compare the validation accuracy of the Sigmoid and the ReLU versions.

$$\mathrm{ReLu} = \begin{cases} \max(0, x) & \text{if } x > 0 \\ 0 & \text{if } x \leqslant 0 \end{cases} \tag{46}$$

To compute the backward operation of the ReLU function, we calculate the gradient with respect to the input $x$. The gradient can easily be calculated as a step function with value 0 when $x \leqslant 0$ and 1 when $x > 0$.

$$\frac{\partial \mathrm{ReLU}}{\partial x} = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leqslant 0 \end{cases} \tag{47}$$

To include this operation in the ops.py file we will define the forward and backward as follows:

```
"""Assignment 2 question 9"""
class ReLU(Op):
    """
    Op for ReLU activation function
    """
    @staticmethod
    def forward(context, x):
        # This line computes a boolean array or mask, which stores the values for which the
        # We expect to see a mask like this Output: # [[ True False False  True False]]
        context['relu'] = x>0
        #Apply the relu function which returns the max between 0 and x value meaning for x-
        # max(0,-2)->0...
        return np.maximum(0, x)

    @staticmethod
    def backward(context, go):

        boolean_values = context['relu']
        # To apply the backward is the multiplication of the gradient times the boolean mas
        return go * boolean_values
```

Once create the ReLU function we include it in the utility functions as follows:

```
def relu(x):
    """
    Wrap tue relu op in a funciton.

    :param x:
    :return:
    """
    return ReLU.do_forward(x)
```

We now compare the validation accuracy of the Sigmoid and the ReLU activation functions for different epochs keeping the rest of training hyperparameters unaltered to evalute their effect in the models performance. For this experiment we used the MNIST dataset.

| **Epochs** | **Sigmoid** | **ReLU** |
|---|---|---|
| 10 | 0.961 | 0.973 |
| 20 | 0.972 | 0.970 |

Table 1: Comparison of Validation Accuracy for Sigmoid and ReLU Nonlinearities using MNIST dataset

As described in table.(1) ReLU activation function exhibits a slight increase in terms of performance for 10 epochs compared to the Sigmoid function. When increasing to 20 epochs the gap between both functions narrows, with no clear advantage between both functions.

**question 10** Change the network architecture (and other aspects of the model) and show how the training behavior changes for the MNIST data.

The default configuration of the model consists of two layers. First hidden layer that applies a linear operation between the inputs and weights followed by nonlinearity function (Sigmoid or Relu), and a second layer which processes the output of the first layer, to produce the output which is then transformed into a probability vector using the softmax function.
hm = hidden-multiplier it specifies the hidden layer size, when increasing hm we will increase the width of the hidden layer.

| **Experiment** | **Parameter Changed** | **Configuration** | **Epochs** | **Validation Accuracy (%)** |
|---|---|---|---|---|
| **Benchmark** | **Default Parameters** | **Default Configuration** | 20 | 0.972 |
| 1 | Layer width | 1 hidden layers (hm = 8) | 20 | 0.974 |
| 2 | Number of Layers | 2 hidden layers (hm = 4) | 20 | 0.960 |
| 4 | Momentum | value = 0.9 | 20 | 0.979 |
| 5 | Momentum | value = 0.8 | 20 | 0.979 |
| 6 | Weights | He | 20 | 0.962 |
| 7 | Weights | Zeros | 20 | 0.135 |
| 8 | Weights | normal distribution | 20 | 0.891 |

Table 2: Validation Accuracy for Different Neural Network Configurations on MNIST. The benchmark row represents the model's default parameters.

To show how the training behaviour changes using different parameters, we can use the training loss over epochs.

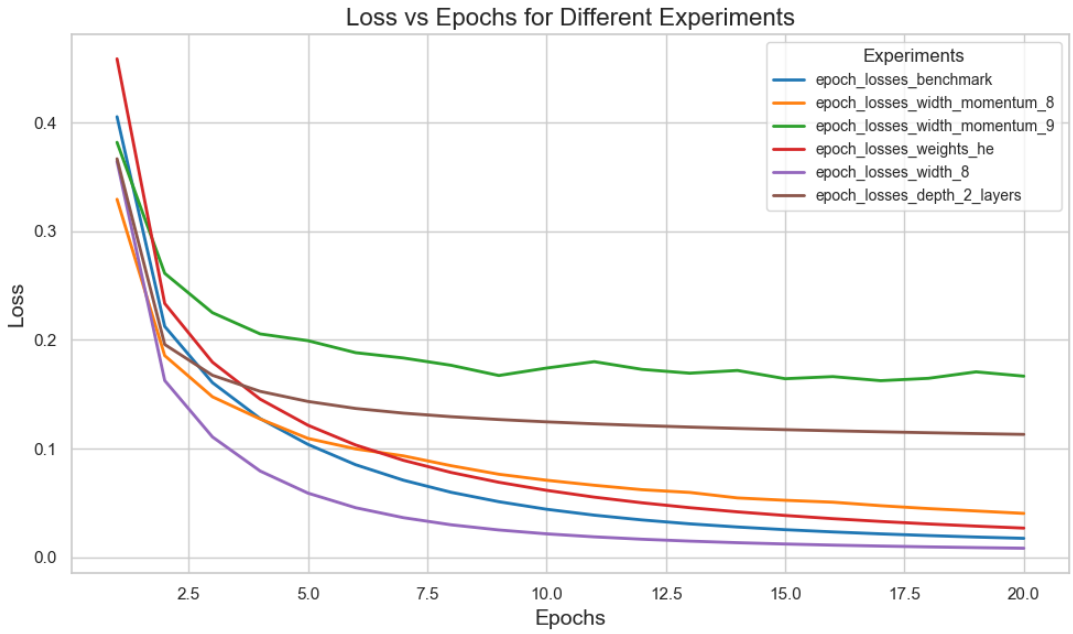


Figure 1: Training loss for different parameters configuration using MNIST dataset

Figure (1) shows loss over epochs for various parameter configurations in the network. When applying a momentum of 0.9, the performance is reduced compared to other configurations. A slower convergence, with minimal decrease over epochs, could be attributed to the high momentum. This likely causes a significant portion of the previous update to influence the current learning step, potentially leading to overshooting the optimal weight updates and, slower convergence. Similarly, the use of two hidden layers also results in a slower convergence rate, possibly due to the increased complexity of the network and the associated challenges in optimizing a deeper architecture. For the other configurations there is a similar convergence pattern, reaching a after 20 epochs minimum when doubling the hidden multiplier from 4 to 8, which indicates how many times bigger the hidden layer is than the input layer. This result suggests that for the given model configuration it is more adequate to increase the width i.e the number of neurons rather than including a second layer which lowers significantly convergence rate.

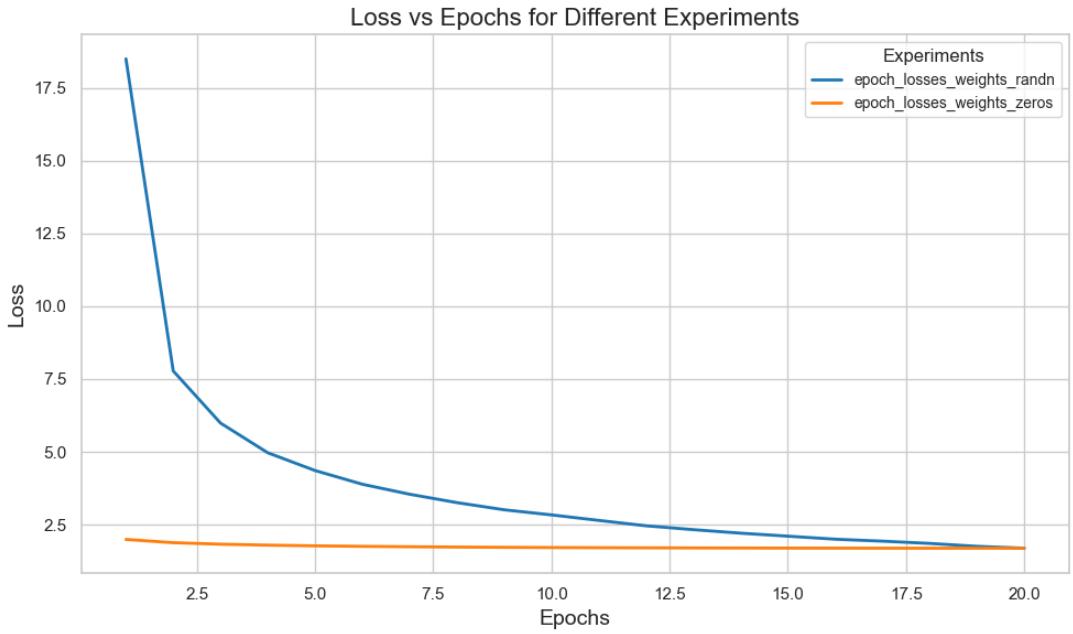


Figure 2: Training loss for different weights initialization using MNIST dataset

Figure.(3), describes the training loss when setting the model with different initialization weights. When setting with the weights with a random distribution, loss starts at higher values for the initial epochs followed by an increased convergence. Contrary when setting the weights as zeros there is no clear learning pattern, which remains flat throughout epochs.

**question 11** When you've built a classifier, play around with the hyperparameters (learning rate, batch size, nr of epochs, etc) and see what the best accuracies are that you can achieve. Report your hyperparameters and your results.

For this section, we will build a Convolutional Neural Network trained using the CIFAR10 dataset, which contains 50,000 training images and 10,000 test images. Each image has dimensions $(3 \times 32 \times 32)$, meaning images in 3 channels.

Following are the steps to build and train the model

1. Load and normalize dataset
2. Define the model

```
class Net(nn.Module):
"""
A Convolutional Neural Network (CNN) for image classification
```

```
    Architecture:
        - Conv Layer 1: 3 input channels, 6 output channels, kernel size 5x5
        - Max Pooling: kernel size 2x2, stride 2
        - Conv Layer 2: 6 input channels, 16 output channels, kernel size 5x5
        - Max Pooling: kernel size 2x2, stride 2
        - Fully Connected Layer 1: Input size 16*5*5, output size 120
        - Fully Connected Layer 2: Input size 120, output size 84
        - Fully Connected Layer 3: Input size 84, output size 10 (number of classes)

    Forward Pass:
        1. Apply convolution and ReLU activation (conv1)
        2. Apply max-pooling
        3. Apply convolution and ReLU activation (conv2)
        4. Apply max-pooling
        5. Flatten the feature maps
        6. Pass through three fully connected layers with ReLU activations for the firs
        7. Output for 10 classes
    """
    def __init__(self):
        super().__init__()
        # set first convolution
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1) # flatten all dimensions except batch
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

The main functions used to train and evaluate the model can be found in section (A **Appendix**). The main function is reported below which performs the training and validation in the test dataset.

```
    def run(model=None,train_loader = None,test_loader = None,epochs=2,criterion=No
    """
    Arguments:
        -model
```

```
        -train_loader
        -test_loader
        -epoch
        -criterion
        -optimizer
        -show_sample
        -plot_loss
    Output:
        -loss, accuracy: list containing loss over epochs, scalar value for accuracy
    """
    # If true then show some batch sample
    if show_sample:
        view_samples(train_loader)
    #   Train model
    loss = train_model(epochs = epochs, model=model,criterion=criterion,optimizer=optim
    # Once trained the model evaluate in the test dataset
    accuracy = evaluate(test_loader=test_loader,model = model)
    if plot_loss:
        plot_loss_curve(loss)
    return loss, accuracy
```

3. Evaluate model To evaluate the model we will use the accuracy as main metric, calculated as the number of correct classified instances over the total number of test samples. We will also plot the loss curve over epochs to evaluate the learning process and convergence rate.

We will evaluate different hyper-parameters configurations including (learning= rate, batch size, number of epochs, momentum,) and evaluate the results in terms of accuracy. To run each experiment, we created a json file that contains the training parameter which were modified accordingly for each iteration.

```
 {
"lr": 0.001,
"epochs":20,
"batch_size":4,
"momentum":0.9
}
```

Table 3: Experimental Results on CIFAR-10 with Parameter Variations

| Experiment | Parameter Changed | Value | Unchanged Parameters | Epochs | Test Accuracy (%) |
|---|---|---|---|---|---|
| 1 | Learning Rate | 0.001 | Batch Size: 4, Momentum: 0.9, Epochs: 20 | 20 | 0.614 |
| 2 | Learning rate | 0.01 | Batch Size: 4, Momentum: 0.9, Epochs: 20 | 20 | 0.101 |
| 3 | Learning rate | 0.0001 | Batch Size: 64, Epochs: 20, Momentum: 0.9 | 20 | 0.618 |
| 4 | Batch size | 1 | Learning Rate: 0.001, Momentum: 0.9, Epochs: 20 | 20 | 0.424 |
| 5 | Batch size | 16 | Learning Rate: 0.001, Momentum: 0.9, Epochs: 20 | 20 | 0.652 |
| 5 | Batch size | 32 | Learning Rate: 0.001, Momentum: 0.9, Epochs: 20 | 20 | 0.6455 |
| 6 | Momentum | 0.8 | Learning Rate: 0.001, Batch size: 4 ,Epochs: 20 | 20 | 0.6197 |
| 7 | Momentum | 0 | Learning Rate: 0.001, Batch size: 4 ,Epochs: 20 | 20 | 0.586 |
| 8 | Epochs | 5 | Learning Rate: 0.001, Batch size: 4, momentum: 0.9 | 5 | 0.485 |
| 9 | Epochs | 10 | Learning Rate: 0.001, Batch size: 4, momentum: 0.9 | 10 | 0.549 |

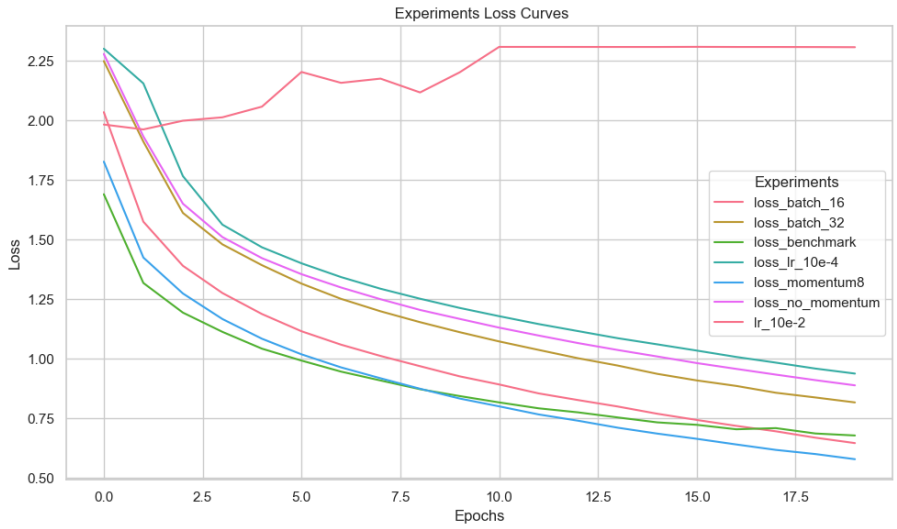


Figure 3: Training loss curve for different hyperparameter configurations

Based on the results, the parameters that significantly impact the model's performance are the number of epochs and the learning rate. The lowest accuracy was observed in Experiment 2, as shown in Table (3). Increasing the number of epochs led to a noticeable improvement in performance, particularly from 1 to 5 epochs, with diminishing returns and slower convergence in subsequent epochs. The highest accuracy was achieved with batch sizes of 16 and 32, while the other parameters were kept at their benchmark values. Contrary when implementing batch size equals 1 as stochastic gradient descent, the accuracy reduce significantly, this might be due to high variance in This does not represent the overall distribution of the data, hence the model learning is highly variable due to the intrinsic variance in the dataset.

**question 12** Change some (at least one, if large enough of a change. T o be safe try three), other aspects of the training and report the results.

Table 4: Experimental Results on CIFAR-10 with CNN Variations

| Experiment | Parameter Changed | Value | Epochs | Test Accuracy (%) |
|---|---|---|---|---|
| | | **Benchmark (Default Configuration)** | | |
| - | Baseline Configuration | Loss: CrossEntropy, Optimizer: SGD, No Batch Norm, No Residual Connections | 20 | 0.685 |
| 1 | Batch Normalization | Added in both conv layers | 20 | 0.653 |
| 2 | Optimizer | RMSprop | 20 | 0.651 |
| 3 | Optimizer | Adam | 20 | 0.644 |
| 4 | Optimizer | Adam + L2 (weight decay: 1e-4) | 20 | 0.659 |
| 5 | Optimizer | SGD + momentum + L2 (weight decay: 1e-4) | 20 | 0.657 |
| 6 | Architecture (number of filters / conv layer) | Double filters in both conv layers (conv1 6− >12, conv2 16 − > 32) | 20 | 0.687 |
| 7 | Architecture(number of linear units) | Linear layer (Double units Linear1 120− >240, Linear2 84− >168) | 20 | 0.649 |
| 8 | Architecture(exp 6) + Adam (L2)optimizer | Batch normalization + Adam (L2) + Double conv filters | 20 | 0.647 |
| 9 | Architecture (Conv layers, Kernel size, number filters) | BN, Adam(L2),Conv layers=3, kernel size=3, filters= 32,64,128 | 20 | 0.764 |
| 10 | Architecture(exp 9) + Optimizer | Batch normalization + **AdamW(L2)** + Double conv filters + third conv layer conv3 | 20 | 0.772 |
| 11 | Loss function | Architecture(exp 9)+ KL Divergence loss | 20 | 0.768 |

Based on the results reported in Table 4, the accuracy of the model can be improved by increasing the number of filters in the convolutional layers. Additionally, adding a third convolutional layer further enhances model performance, as increasing the depth allows the model to learn more complex functions and capture finer details from the input dataset. However, it is important to note the increase in computational complexity and resource requirements when training models with additional layers and filters. Experiment 12 demonstrates the use of KL-divergence as a loss function. In this case, the model's output is treated as a probability distribution and compared to the target distribution represented by the one-hot-encoded labels.

1. Batch normalization
   Add a 2d batch normalization operation after each convolution operation

```
self.conv1 = nn.Conv2d(3, 6, 5)
self.bn1 = nn.BatchNorm2d(6)  # Batch normalization for 6 channels
self.pool = nn.MaxPool2d(2, 2)
self.conv2 = nn.Conv2d(6, 16, 5)
self.bn2 = nn.BatchNorm2d(16)
```

2. Optimizer (RMSprop) Set RMSprop optimizer with default parameters

```
optimizer = optim.RMSprop(nn_model.parameters(),
lr=0.001, alpha=0.9,
weight_decay=1e-4,
momentum=0.9)
```

3. Optimizer (Adam default values)

```
optimizer = optim.Adam(nn_model.parameters())
```

4. Optimizer (Adam, L2 weight decay)

```
optimizer = optim.Adam(nn_model.parameters(), lr=0.001, weight_decay=1e-4)
```

5. Optimizer (SGD, momentum, L2 decay )

```
optimizer = optim.SGD(nn_model.parameters(),
lr=0.001, momentum=0.9,
weight_decay=1e-4)
```

6. Architecture (2 conv layers, double filters in both conv layers )

```
self.conv1 = nn.Conv2d(3, 12, kernel_size=3, padding=1)  # Input: (3, 32, 32) -
self.bn1 = nn.BatchNorm2d(12)
self.conv2 = nn.Conv2d(12, 32, kernel_size=3, padding=1)  # Input: (32, 32, 32)
self.bn2 = nn.BatchNorm2d(32)
```

7. Architecture (Linear layers , number of linear units*2 )

```
self.fc1 = nn.Linear(32 * 5 * 5, 240)
self.fc2 = nn.Linear(120, 160)
self.fc3 = nn.Linear(160, 10)
```

8. Architecture (exp 6) + Adam optimizer

9. Architecture (Conv layers)

```
self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
# Input: (3, 32, 32) -> Output: (32, 32, 32)
self.bn1 = nn.BatchNorm2d(32)
self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
# Input: (32, 32, 32) -> Output: (64, 32, 32)
self.bn2 = nn.BatchNorm2d(64)
self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
# Input: (64, 16, 16) -> Output: (128, 16, 16)
self.bn3 = nn.BatchNorm2d(128)
```

10. Architecture (exp 9 )+AdamW

```
optimizer = optim.AdamW(nn_model.parameters(), lr=0.001, weight_decay=1e-4)
```

11. Using KL divergence loss
    To use KL divergence we must change the output of the model to a probability vector using softmax function and provided labels should be one hot encoded

```
# Set softmax in forward function
x = F.log_softmax(x, dim=1)
```

    In the training method convert the labels batch into one-hot encoding vectors

```
for i, data in enumerate(data_loader, 0):
    # Unpack data into inputs and labels
    inputs, labels = data
    batch_size = len(inputs)
    one_hot_labels = F.one_hot(labels, num_classes=10).float()
    # Forward pass
    outputs = model(inputs)

    # Calculate loss
    loss = criterion(outputs, one_hot_labels)
```

Lastly, we obtained the train loss curve for all the experiments.

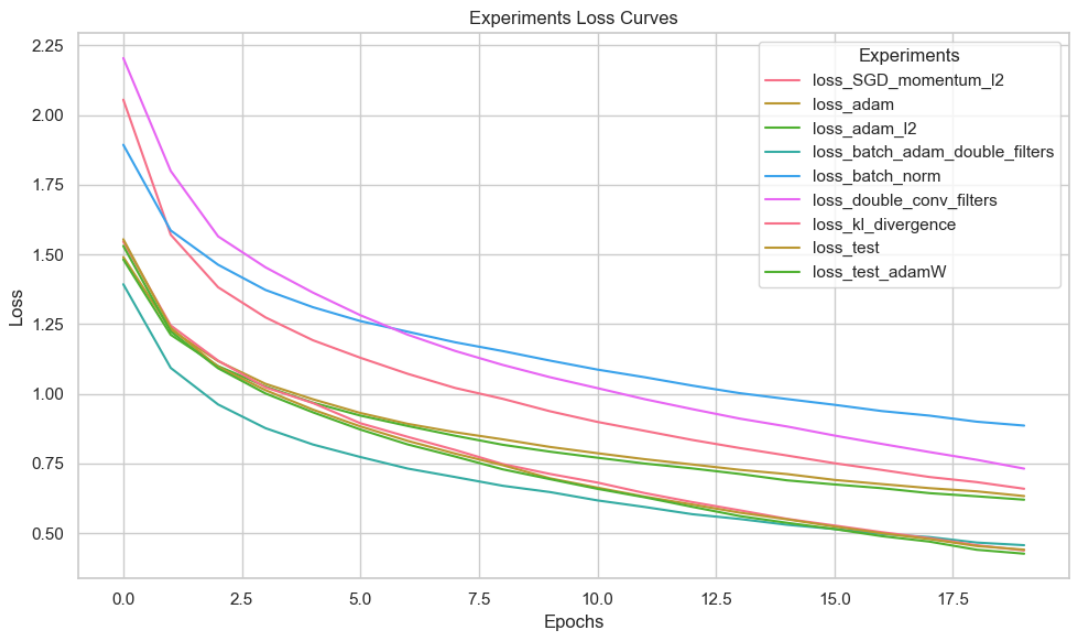


Figure 4: Training loss curve for different model architectures, optimizers and regularization techniques

# A Appendix

**Question 6.2** Tracing the addition operation

1. Operation

```
c = a+b
```

2. File: core.py, Call method doforward()

```
class TensorNode:
    def __add__(self, other):
        return Add.do_forward(self, other)
```

3. Move to abstract class Op

```
class Op:
    def do_forward()
        outputs_raw = cls.forward(context, *inputs_raw, **kwargs)
```

4. Move the child of Op class Ad(Op) which implements the abstract method forward

```
class Add(Op)
    def forward(context,a,b)
        return a + b
```